

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 20%

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1. Real-Time Face Detection and Recognition Pipeline

A system must process a live video stream to:

- A. Capture frames from a camera
- B. Detect faces in each frame
- C. Recognize faces using a pre-trained dataset
- D. Display bounding boxes and labels in real time

Constraints:

- Must operate continuously
- Should minimize processing delay
- Should handle multiple faces

Write detailed pseudocode for the complete pipeline using OpenCV concepts, including image acquisition, preprocessing, detection, recognition, and display. Ensure modular design and efficient looping structure.

2. OCR Pipeline for Document Processing

A document processing system must:

- A. Read input images containing printed text
- B. Enhance image quality (noise removal, contrast improvement)
- C. Segment text regions
- D. Extract and display recognized text

Constraints:

- Input images may be noisy and low contrast
- Batch processing required

Write pseudocode for an end-to-end OCR pipeline using OpenCV functions. Clearly structure preprocessing, segmentation, and text extraction stages.

4. Vehicle Counting System using Video

A traffic monitoring system must:

- A. Read video frames
- B. Detect vehicles entering a region of interest (ROI)
- C. Track movement across frames
- D. Count vehicles and display count

Constraints:

- Avoid double counting
- Handle multiple vehicles simultaneously

Write pseudocode for the complete system, including detection, tracking, counting logic, and visualization using drawing functions.

5. Facial Expression Recognition System

A mobile-based system must:

- A. Capture images from a camera
- B. Detect faces
- C. Extract facial features
- D. Classify expressions (happy, sad, neutral)
- E. Display results

Constraints:

- Limited computational resources
- Real-time response required

Write pseudocode for the pipeline, ensuring efficient processing and modular design for detection, feature extraction, and classification.

9. Background Subtraction Pipeline

A surveillance system must:

- A. Read video stream
- B. Separate foreground from background
- C. Highlight moving objects

Constraints:

- Handle dynamic background

- Reduce noise

Write pseudocode including background modeling, subtraction, filtering, and display.

11. Multi-Image Batch Processing System

A system must:

- A. Read multiple images from a folder
- B. Apply preprocessing and feature detection
- C. Store results

Constraints:

- Batch processing
- Efficient memory usage

Write pseudocode with modular structure and looping logic.

12. Face Detection with ROI Extraction

A system must:

- A. Detect faces in an image
- B. Extract face regions
- C. Save cropped faces

Constraints:

- Handle multiple faces
- Avoid redundant processing

Write pseudocode including detection, ROI extraction, and saving logic.

14. Text Detection in Natural Scene Images

A system must:

- A. Read images
- B. Detect text regions
- C. Highlight detected areas

Constraints:

- Handle varying backgrounds
- Improve detection accuracy

Write pseudocode including preprocessing, region detection, and visualization.

Code of Conduct:

*I K.ASWINI(192312397)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized	Handles most cases;	Handles basic cases only	Incomplete or inefficient

		approach	reasonable efficiency		
--	--	----------	-----------------------	--	--

ANSWERS

1. Real-Time Face Detection & Recognition Pipeline

Modules

- Frame Capture
- Preprocessing
- Face Detection
- Face Recognition
- Display

PSEUDO CODE :

```

INITIALIZE camera using VideoCapture(0)
LOAD face_detector (Haar Cascade or DNN)
LOAD face_recognizer (LBPH / CNN model)
LOAD label_map (ID → Name)

FUNCTION preprocess(frame):
    gray = convert_to_grayscale(frame)
    resized = resize(gray, fixed_size)
    RETURN resized

FUNCTION detect_faces(frame):
    faces = face_detector.detectMultiScale(frame)
    RETURN faces

FUNCTION recognize_face(face_region):
    label_id, confidence = face_recognizer.predict(face_region)
    name = label_map[label_id]
    RETURN name, confidence

WHILE camera is open:
    ret, frame = camera.read()
    IF ret == FALSE:
        BREAK

    processed_frame = preprocess(frame)

    faces = detect_faces(processed_frame)

    FOR each (x, y, w, h) in faces:
        face_roi = processed_frame[y:y+h, x:x+w]

```

```

        name, confidence = recognize_face(face_roi)

        DRAW rectangle(frame, (x,y), (x+w,y+h))
        PUT_TEXT(frame, name + confidence, position=(x,y-10))

    DISPLAY frame

    IF key_pressed == 'q':
        BREAK

RELEASE camera
DESTROY all windows

```

2. OCR Pipeline for Document Processing

Modules

- Image Loading (Batch)
- Preprocessing
- Text Segmentation
- OCR Extraction

PSEUDO CODE :

```

IMPORT OCR engine (Tesseract)

FUNCTION preprocess(image):
    gray = convert_to_grayscale(image)
    denoise = gaussian_blur(gray)
    thresh = adaptive_threshold(denoise)
    contrast = histogram_equalization(thresh)
    RETURN contrast

FUNCTION segment_text(image):
    contours = findContours(image)
    text_regions = []
    FOR each contour:
        IF contour_area > threshold:
            bbox = bounding_rect(contour)
            ADD bbox to text_regions
    RETURN text_regions

FUNCTION extract_text(region):
    text = OCR_engine(region)
    RETURN text

LOAD all images from folder

FOR each image_path in folder:
    image = read_image(image_path)

    processed = preprocess(image)

    regions = segment_text(processed)

```

```

full_text = ""

FOR each region in regions:
    roi = crop(processed, region)
    text = extract_text(roi)
    full_text += text

PRINT full_text
SAVE text to file

END

```

4. Vehicle Counting System using Video

Modules

- Detection
- Tracking
- Counting Logic

PSEUDO CODE :

```

INITIALIZE video capture
LOAD vehicle_detector (YOLO / Haar)
INITIALIZE tracker (Centroid Tracker)
SET ROI_line_position
SET vehicle_count = 0

FUNCTION detect_vehicles(frame):
    detections = vehicle_detector.detect(frame)
    RETURN bounding_boxes

FUNCTION track_objects(detections):
    tracked_objects = tracker.update(detections)
    RETURN tracked_objects

FUNCTION count_logic(tracked_objects):
    FOR each object_id, centroid:
        IF centroid crosses ROI_line AND object not counted:
            vehicle_count += 1
            mark object as counted

WHILE video is running:
    frame = read_frame()

    detections = detect_vehicles(frame)

    tracked_objects = track_objects(detections)

    count_logic(tracked_objects)

    DRAW ROI line
    DRAW bounding boxes + IDs

    DISPLAY vehicle_count on frame

```

```
SHOW frame

IF 'q' pressed:
    BREAK

RELEASE video
```

5. Facial Expression Recognition System

Modules

- Detection
- Feature Extraction
- Classification

PSEUDO CODE :

```
INITIALIZE camera
LOAD face_detector
LOAD expression_model (CNN lightweight)

FUNCTION preprocess(face):
    resized = resize(face, 48x48)
    normalized = normalize(resized)
    RETURN normalized

FUNCTION classify_expression(face):
    prediction = expression_model.predict(face)
    label = argmax(prediction)
    RETURN label

WHILE camera is running:
    frame = capture_frame()

    gray = convert_to_grayscale(frame)

    faces = detect_faces(gray)

    FOR each face:
        roi = extract_face(gray)

        processed = preprocess(roi)

        emotion = classify_expression(processed)

        DRAW rectangle
        DISPLAY emotion label

    SHOW frame

    IF 'q':
        BREAK

RELEASE resources
```

9. Background Subtraction Pipeline

Modules

- Background Modeling
- Subtraction
- Noise Reduction

PSEUDO CODE :

```
INITIALIZE video capture
CREATE background_subtractor (MOG2/KNN)

FUNCTION clean_mask(mask):
    mask = morphological_open(mask)
    mask = morphological_close(mask)
    RETURN mask

WHILE video is running:
    frame = read_frame()

    fg_mask = background_subtractor.apply(frame)

    clean = clean_mask(fg_mask)

    contours = findContours(clean)

    FOR each contour:
        IF area > threshold:
            DRAW bounding box

    DISPLAY original frame
    DISPLAY foreground mask

    IF 'q':
        BREAK

RELEASE resources
```

11. Multi-Image Batch Processing System

Modules

- Batch Loader
- Preprocessing
- Feature Extraction

PSEUDO CODE :

```
LOAD image_paths from folder

FUNCTION preprocess(image):
    gray = convert_to_grayscale(image)
    blur = gaussian_blur(gray)
    RETURN blur

FUNCTION detect_features(image):
    keypoints = ORB_detect(image)
    descriptors = ORB_compute(image)
    RETURN keypoints, descriptors

FOR each image_path:
    image = read_image(image_path)

    processed = preprocess(image)

    kp, desc = detect_features(processed)

    SAVE descriptors to file
    OPTIONAL: draw keypoints and save image

    RELEASE memory for image

END
```

12. Face Detection with ROI Extraction

PSEUDO CODE :

```
LOAD face_detector
READ input image

faces = detect_faces(image)

FOR i, (x,y,w,h) in faces:
    roi = image[y:y+h, x:x+w]

    filename = "face_" + i + ".jpg"

    SAVE roi

    DRAW rectangle on original image

DISPLAY result image
```

14. Text Detection in Natural Scene Images

Modules

- Preprocessing
- Region Detection (EAST/Contours)
- Visualization

- PSEUDO CODE :

```
LOAD text_detector (EAST model)

FUNCTION preprocess(image):
    resized = resize(image)
    blob = create_blob(resized)
    RETURN blob

FUNCTION detect_text(image):
    blob = preprocess(image)
    detections = EAST_forward(blob)
    boxes = decode_predictions(detections)
    RETURN boxes

READ input image

boxes = detect_text(image)

FOR each box:
    DRAW rectangle around text region

DISPLAY image
SAVE output
```